المصفوفة ثنائية الأبعاد

Two-Dimensional Array

من الصف إلى الجدول – تنظيم البيانات في صفوف وأعمدة

4	8	3	1
7	2	9	5
6	11	14	2

إعداد: الأستاذ محمود اغبارية

⊞ ما هي المصفوفة ثنائية الأبعاد؟

المصفوفة الأحادية تُشبه **صفاً واحداً** من الخانات. المصفوفة ثنائية الأبعاد تُشبه **جدولاً** – لها صفوف (rows) وأعمدة (columns).

مصفوفة أحادية (5 عناصر)

10	20	30	40	50

→

مصفوفة ثنائية (4×3)

1	2	3	4
5	6	7	8
9	10	11	12

💡 **التعريف:** المصفوفة ثنائية الأبعاد هي **مصفوفة من المصفوفات** – كل عنصر فيها هو صف (مصفوفة أحادية).

الإعلان والتهيئة

1. إعلان فارغ ثم تخصيص:

```
int[,] mat;
mat = new int[3, 4]; // 3 أعمدة، 4 صفوف
```

2. إعلان وتخصيص في سطر واحد:

```
int[,] mat = new int[3, 4];
```

3. إعلان بقيم مباشرة:

```
int[,] mat = {
  { 1,  2,  3,  4 },
  { 5,  6,  7,  8 },
  { 9, 10, 11, 12 }
};
```

الجدول الناتج:

	[0]	[1]	[2]	[3]
[0]	1	2	3	4
[1]	5	6	7	8
[2]	9	10	11	12

عدد الصفوف: `mat.GetLength(0)` → 3

عدد الأعمدة: `mat.GetLength(1)` → 4

الوصول إلى العناصر

للوصول إلى عنصر نكتب: `mat[row, col]` – صف أولاً، ثم عمود.

	[0]	[1]	[2]	[3]
[0]	1	2	3	4
[1]	5	6	7	8
[2]	9	10	11	12

```
// قراءة
int x = mat[1, 2]; // = 7

// كتابة
mat[0, 3] = 99;
```

انقر على أي خلية لمعرفة فهرسها:

⚠ **تنبيه:** الفهارس تبدأ من 0. الصف 3 في مصفوفة 3 صفوف → `IndexOutOfRangeException`!

المرور على المصفوفة – حلقات متداخلة

```
for (int i = 0; i < mat.GetLength(0); i++)
{
  for (int j = 0; j < mat.GetLength(1); j++)
  {
    // معالجة mat[i, j]
    Console.Write(mat[i, j] + " ");
  }
  Console.WriteLine();
}
```

i = رقم الصف | j = رقم العمود

الحلقة الخارجية تمر على الصفوف، الداخلية على الأعمدة.

اضغط "التالي" لبدء المرور خلية خلية

- = i | - = j | - = mat[i,j]

1	2	3	4
5	6	7	8
9	10	11	12

السابق 1 / 1 التالي

تمرين 1

معطاة المصفوفة:

1	2	3
4	5	6
7	8	9

1. ما قيمة `mat[2, 1]`؟
2. ما الفهرس `[row, col]` للقيمة 6؟
3. ما ناتج `mat.GetLength(0)`؟ وما ناتج `mat.GetLength(1)`؟
4. أكتب سطر كود يضع القيمة 99 في الزاوية اليمنى السفلى.

إجابة تمرين 1

1. `mat[2, 1]` = **8** (الصف 2، العمود 1)
2. القيمة 6 في الفهرس **[2 ,1]** (الصف 1، العمود 2)
3. `mat.GetLength(0)` = **3** (عدد الصفوف) | `mat.GetLength(1)` = **3** (عدد الأعمدة)
4.
```
mat[2, 2] = 99;
```
لأن الزاوية السفلى اليمنى هي الفهرس `[rows-1, cols-1] = [2, 2]`

تمرير المصفوفة لعملية خارجية

نُمرِّر المصفوفة ثنائية الأبعاد تماماً كالأحادية – نكتب النوع مع `[,]`:

```
// عملية تحسب مجموع كل عناصر المصفوفة
public static int Sum(int[,] mat)
{
    int total = 0;
    for (int i = 0; i < mat.GetLength(0); i++)
        for (int j = 0; j < mat.GetLength(1); j++)
            total += mat[i, j];
    return total;
}

// الاستدعاء:
int s = Sum(mat);
```

ملاحظات مهمة:

- المصفوفة تُمرَّر **بالمرجع** – أي تعديل داخل العملية يؤثر على الأصل.
- لا نحتاج لتمرير أبعاد المصفوفة – نستخدم `GetLength()` داخل العملية مباشرةً.

مقارنة مع الأحادية:

`int[] arr` → `int[] arr`

`int[,] mat` → `int[,] mat`

محاكاة: حساب مجموع كل صف

```
public static void PrintRowSums(int[,] mat)
{
  for (int i=0; i<mat.GetLength(0); i++)
  {
    int sum = 0;
    for (int j=0; j<mat.GetLength(1); j++)
      sum += mat[i, j];
    Console.WriteLine("Row "+i+": "+sum);
  }
}
```

اضغط "التالي" لبدء حساب مجموع كل صف

- = sum | - = j | - = i

2	4	1
3	5	2
1	3	4

السابق | 1 / 1 | التالي

↺ إعادة مصفوفة ثنائية من عملية خارجية

نكتب النوع المُعاد `int[,]` قبل اسم العملية، ونستخدم `return` كالمعتاد:

```
// عملية تُنشئ مصفوفة هويّة (1 على القطر، 0 في بقية المواضع)
public static int[,] CreateIdentity(int n)
{
    int[,] mat = new int[n, n];
    for (int i = 0; i < n; i++)
        mat[i, i] = 1;          // القطر الرئيسي
    return mat;
```

نفس الفكرة كالمصفوفة الأحادية:

- نُنشئ المصفوفة داخل العملية بـ `new`
- نملأها بالقيم المطلوبة
- نُعيدها بـ `return`

الناتج CreateIdentity(3):

1	0	0
0	1	0
0	0	1

محاكاة: عملية تُنشئ نسخة من المصفوفة

```
public static int[,] CopyMatrix(int[,] src)
{
  int rows = src.GetLength(0);
  int cols = src.GetLength(1);
  int[,] dst = new int[rows, cols];

  for (int i=0; i<rows; i++)
    for (int j=0; j<cols; j++)
      dst[i,j] = src[i,j];

  return dst;
}
```

في المحاكاة: كل خلية تُنسخ من src إلى dst واحدةً تلو الأخرى.

اضغط "التالي" لمشاهدة عملية النسخ خلية بخلية

src				dst		
7	2	5	→	·	·	·
1	8	3		·	·	·
4	6	9		·	·	·

السابق 1 / 1 التالي

تمرين 2 – كتابة كود

اكتب عملية خارجية `FindMax` تتلقى مصفوفة أعداد صحيحة ثنائية الأبعاد `mat`، وتُعيد أكبر قيمة فيها.

3	7	1
9	2	5
4	8	6

→ تُعيد 9

إجابة تمرين 2

```
public static int FindMax(int[,] mat)
{
    int max = mat[0, 0]; // نبدأ بافتراض أن أول عنصر هو الأكبر

    for (int i = 0; i < mat.GetLength(0); i++)
        for (int j = 0; j < mat.GetLength(1); j++)
            if (mat[i, j] > max)
                max = mat[i, j];

    return max;
}
```

الفكرة: نفس فكرة إيجاد الأكبر في مصفوفة أحادية – لكن بحلقتين متداخلتين بدلاً من واحدة.

? تمرين 3 – عملية تُعيد مصفوفة

اكتب عملية خارجية `Transpose` تتلقى مصفوفة أعداد صحيحة ثنائية الأبعاد **مربّعة** (n×n)، وتُعيد **منقولتها** – أي تبديل الصفوف بالأعمدة.

المصفوفة الأصلية

1	2	3
4	5	6
7	8	9

→

بعد النقل

1	4	7
2	5	8
3	6	9

لاحظ: العنصر الذي كان في `[i, j]` أصبح في `[j, i]`.

إجابة تمرين 3

```
public static int[,] Transpose(int[,] mat)
{
    int n = mat.GetLength(0); // مصفوفة مربّعة → نفس البُعدين
    int[,] result = new int[n, n];

    for (int i = 0; i < n; i++)
        for (int j = 0; j < n; j++)
            result[j, i] = mat[i, j]; // نقلب الفهارس

    return result;
}
```

المفتاح: السطر `result[j, i] = mat[i, j]` – ببساطة نعكس الفهرسين عند النسخ.

❓ تمرين 4 – عملية تُعيد مصفوفة أحادية

اكتب عملية خارجية `RowSums` تتلقى مصفوفة أعداد صحيحة ثنائية الأبعاد `mat`، وتُعيد **مصفوفة أحادية** بحيث يحتوي العنصر `i` على مجموع عناصر الصف `i` في المصفوفة الثنائية.

mat (3×4)

1	2	3	4
5	6	7	8
9	10	11	12

→

النتيجة (مصفوفة أحادية)

10	← 1+2+3+4
26	← 5+6+7+8
42	← 9+10+11+12

إجابة تمرين 4 – مع محاكاة

```
public static int[] RowSums(int[,] mat)
{
    int rows = mat.GetLength(0);
    int cols = mat.GetLength(1);
    int[] sums = new int[rows];

    for (int i = 0; i < rows; i++)
        for (int j = 0; j < cols; j++)
            sums[i] += mat[i, j];

    return sums;
}
```

اضغط "التالي" لمشاهدة بناء المصفوفة الأحادية

- = sums[i] - = j - = i

mat → sums[]

1	2	3	4
5	6	7	8
9	10	11	12

0
0
0

السابق 1 / 1 التالي

نقاط مهمة:

- العملية تُعيد `int[]` (أحادية) وليس `int[,]`
- حجم المصفوفة المُعادة = عدد الصفوف فقط
- `sums[i]` يتراكم مع كل عنصر في الصف `i`

ملخص الدرس

الإعلان

```
int[,] mat = new int[r, c];
```

الوصول

```
mat[i, j]
```

الأبعاد

GetLength(0) → صفوف

GetLength(1) → أعمدة

التمرير

```
Method(int[,] mat)
```

الإعادة

```
public static int[,] Method( ... )
```

المرور

حلقتان متداخلتان – خارجية للصفوف، داخلية للأعمدة

أرجو لكم عملاً ممتعاً! / الأستاذ محمود اغبارية